ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Institute for
Dynamic Systems and Control
IDSC
Institut für Dynamische Systeme
und Regelungstechnik

William Akuffo

On the Design of Terminal Ingredients for MPCC (using RL)

Master Thesis

Institute for Dynamic Systems and Control
Swiss Federal Institute of Technology (ETH) Zurich

Supervision

Dr. Andrea Carron
Prof. Dr. Melanie Zeilinger

December 2024

IDSC-XX-YY-ZZ

Contents

Abstract **iii**

Nomenclature **1**

1 Introduction **2**
1.1 Related Works . . . 3
1.2 Contribution . . . 3
1.3 Report Structure . . . 3

2 Preliminaries **5**
2.1 Actor-Critic Reinforcement Learning . . . 5
2.2 MPCC for Autonomous Racing . . . 6
2.2.1 Reference Path . . . 7
2.2.2 Contouring error . . . 7
2.3 Pacejka Dynamic Curvilinear Bicycle Model . . . 7
2.4 Path Contraints . . . 8

3 Approach **10**
3.1 General Overview . . . 10
3.2 MPCC warm start using RL Policy . . . 10
3.2.1 Warm-start using an initial guess and dynamics . . . 11
3.2.2 Warm-start using an Actor (RL policy) and dynamics . . . 12

4 Results **13**
4.1 MPCC Policy with RL Terminal Cost . . . 13
4.1.1 Short horizon, N = 20 . . . 13
4.1.2 Long horizon, N = 30 . . . 16
4.1.3 Race Track Velocity Map . . . 16
4.2 Computational Impact . . . 17

5 Conclusion **21**

Acknowledgement

The success of this work would not have been possible without my supervisor, Dr. Andrea Carron for his guidance and feedback. For his time and dedication to my supervision.
In second place, Manish Prajapat for his time and advice on this work when it became relevant.

Abstract

Model Predictive Contouring Control (MPCC) is a variant of Model Predictive Control (MPC) typically applied to the context of autonomous racing. The nature of the cost function is such that it minimizes an orthogonal deviation from the reference path and maximizes progress along that path. Depending on the penalties on the progress and error terms, an optimal racing line can be solved for implicitly and followed by the racing car. In autonomous racing, a high performance controller is required to satisfy the objectives of staying within track limits and minimizing lap time. Since this optimization problem cannot be solved for the entire duration of the race due to practical constraints, it is important to formulate a Finite Time Optimal Control Problem that includes terminal ingredients to capture the remainder of the infinite horizon. In Actor-Critic Reinforcement Learning, the Critic learns a value function analogous to the terminal cost-to-go required in optimal control. This value function is used to approximate the terminal cost in the MPCC optimal control problem, to which its influence on the resulting control policy is investigated in terms of the planning horizon length of the controller. Finally, a case for warm starting the MPCC controller with the Actor (RL policy) in hard to guess input regions on the racetrack such as a chicane is made.

Nomenclature

Acronyms and Abbreviations

MPCC	Model Predictive Contouring Control
RL	Reinforcement Learning
ANN	Artificial Neural Network
ACRL	Actor-Critic Reinforcement Learning
SQP	Sequential Quadratic Program
KKT	Karush-Kuhn-Tucker

Chapter 1

Introduction

Model Predictive Contouring Control (MPCC), a variant of Model Predictive Control (MPC), is increasingly gaining popularity in recent times. In the context of autonomous racing, it provides a unique advantage in maximizing progress along the racing path as well as minimizing the orthogonal deviation from this reference path [2, 4]. The problem formulation uniquely has the potential to allow for the planning of an optimal racing line along the race track. Depending on the penalties in the cost function, optimal driving behavior can be planned[4]. This makes it a flexible and desirable approach for autonomous racing.
Generally, in Optimal Control, an optimization problem is solved often in the form of a minimization problem of some cost function subject to the constraints of the dynamics, desirable states, the initial condition, and the limitations of the control or actuated input [9]. If it were possible, it would be desirable to solve the problem to capture the entire duration of the task in an infinite-horizon fashion. However, one can quickly imagine the impracticability of accounting for infinite decision variables and constraints. To work around this, a tractable version of the problem is formulated in the finite sense as a Finite Time Optimal Control Problem, neglecting the remainder of the horizon but accounting for it by including terminal ingredients, namely;

- Terminal Cost function, which is ideal Control Lyapunov:
  $\forall x \in X_N \subseteq X$, $\exists u \in U$ such that $f(x,u) \in X_N \subseteq X$
- Terminal constraint set :
  $\forall x \in \mathcal{X}_N \subseteq \mathcal{X}$, $\exists u \in \mathcal{U}$ such that $f(x,u) \in \mathcal{X}_N \subseteq \mathcal{X}$

If these terminal ingredients are correctly designed, the remainder of the control problem left over from formulating a finite time problem can be captured, closing the optimality gap. It also offers the potential to reduce horizon length and computational overhead for faster computation[6]. This is especially important in autonomous racing, where fast-changing dynamics have to be satisfied by a nimble controller[1, 4]. These terminal ingredients can be especially challenging to design, especially in the nonlinear case. Backward Reachability Analysis [10] and level curves of the Lyapunov function can be used to determine terminal constraint sets

that is potentially maximally stabilizing. The solution to the Algebraic Riccatti equation in the Linear Quadratic Case can be a good fit for the terminal cost.

1.1 Related Works

With policy gradient methods becoming increasingly convincing in solving continuous optimal control problems [3, 12], some learning based approaches like [6, 7] have demostrated how some of these terminal ingredients can be designed from data. Actor-Critic Reinforcement Learning can provide a value function learned by the critic which approximates the similar to the terminal cost [6]. A paper by [5], upon which this work is inspired reveals Actor-Critic Reinforcement Learning in the context of autonomous racing, where a submodular RL approach is proposed to handle non-additive rewards by greedily maximizing marginal gains in order to tackle the diminishing returns on rewards. The racing dynamics are captured by a pacejka curvilinear model with state variables described in the local frame of the race car. This allows the ACRL algorithm learn well. To prevent model mismatch, the same model is employed in this work. Another paper by [4] presents two approaches for autonomous racing based on mathematical optimization. The first uses a high level path planner to generate a feasible racing line tracked by an MPC controller. The second approach uses the centerline as reference, but only as a mere measure of progress. Penalties in the cost function of the Model Predictive Contouring controller can be tuned to find an optimal racing line while satisfying constraints of states and inputs. This allows the driven trajectory to be similar to those driven by expert drivers. .

1.2 Contribution

In this work, a value function to approximate the terminal cost, based on the work of [5] is learned by the Critic of an Actor-Critic Reinforcement Learning algorthim. This is trained offline on a given racetrack. The value function is parameterized by a pytorch neural network and plugged into the MPCC optimization problem using L4CasADi [8] to capture the remainder of the policy. Two main contributions are made.

- We show how the resulting MPCC control policy is influenced by the terminal cost as a function of the planning horizon length through a horizon slider.
- A case for warm starting the MPCC solver at the initial step is made for hard to guess input regions like the chicane on the racetrack.

1.3 Report Structure

The report goes through some preliminaries required for this work in Chapter 2. Chapter 3 reveals a general overview of the approach used in this work. The results

are discussed in Chapter 4 and Chapter 5 closes the report with a summary.

Chapter 2

Preliminaries

This chapter explains the approach used in the work. It elaborates on MPCC as a variant of MPC applied to autonomous racing. Further, it discusses ACRL as a learning based approach for estimating the terminal cost as a value function learned by the critic [6].

2.1 Actor-Critic Reinforcement Learning

In Actor-Critic RL, an agent or an actor within an interactive environment tries to maximize the expected return on rewards. It does this by taking an action a on a given state s in the environment based on some learned policy $\pi_\Omega(a|s)$ parameterized by Ω. See Fig. 2.1. A critic learns a value function $V_w(s)$ that describe how desirable a given state is. In a complementary fashion, the Actor tries to learn a policy to maximize the expected return on rewards by maximizing a policy gradient in Eq. (2.2). This is done through gradient ascent in Eq. (2.3).

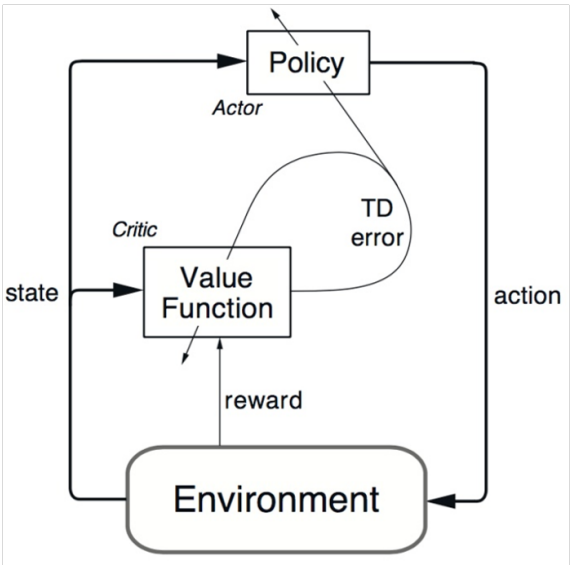


Figure 2.1: Actor-Critic Reinforcement Learning

$$J(\pi_\Omega) = \mathbb{E}_{\pi_\Omega}\left[\sum_{t=0}^{T-1} R(s_t, a_t)\right] \tag{2.1}$$

$$\nabla_\Omega J(\Omega) \approx \frac{1}{N}\sum_{i=1}^{N} \nabla_\Omega \log \pi_\Omega(a_i|s_i) \cdot \delta_i \tag{2.2}$$

$$\Omega_{k+1} = \Omega_k + \alpha \cdot \nabla J(\pi_\Omega) \tag{2.3}$$

The critic on the other hand tries to minimize some temporal difference error and use it as feedback to guide the actor. This is done via gradient descent in Eq. (2.5) of Eq. (2.4). The temporal difference minimized by the critic is described by Eq. (2.6)

$$\nabla_w J(w) \approx \frac{1}{N}\sum_{i=1}^{N} \nabla_w \left(V_w(s_i) - (r_i + \gamma V_w(s_{i+1}))\right)^2 \tag{2.4}$$

$$w_{k+1} = w_k + \alpha \cdot \nabla J(w) \tag{2.5}$$

$$\delta = r + \gamma V(s') - V(s) \tag{2.6}$$

2.2 MPCC for Autonomous Racing

The primary objectives of autonomous racing is to minimize lap time while staying within track limits. A model predictive contouring controller has the ability to combine the path planning and path tracking into one nonlinear optimization problem [4]. The weights in the cost function terms can be tuned to solve for an optimal reference trajectory along the track similar to those driven by racing drivers. The aim of the contouring controller is to maximize path progress while minimizing the contouring error within the constraints posed to the problem.

$$\min_N \sum_{k=1}^{N} \|e_{c,k}(X_k, Y_k, \theta_P)\|_{q_c}^2 - \gamma\theta_{P,N} \tag{2.7}$$

$$\text{s.t.} \quad x_0 = x, \tag{2.8}$$

$$x_{k+1} = f(x_k, u_k), \quad k = 0, \ldots, N-1, \tag{2.9}$$

$$F_k x_k \leq f_k, \quad k = 1, \ldots, N, \tag{2.10}$$

$$\underline{x} \leq x_k \leq \overline{x}, \quad k = 1, \ldots, N, \tag{2.11}$$

$$\underline{u} \leq u_k \leq \overline{u}, \quad k = 0, \ldots, N-1. \tag{2.12}$$

A few preliminaries in the context of MPCC are discussed as follows.

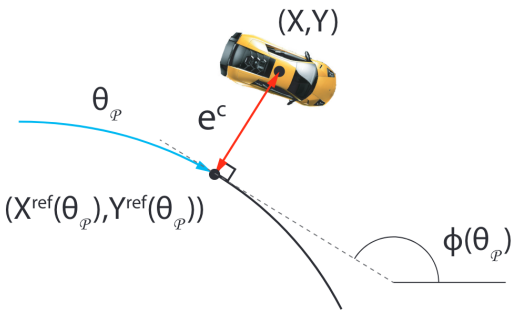


Figure 2.2: Contouring error

2.2.1 Reference Path

The centerline of the racetrack is parameterized by some path parameter θ closed on $[0, L]$ where L is ideally the length of one lap.

2.2.2 Contouring error

This is the orthogonal deviation e_c in Eq. (2.13) from the reference point on the reference path.

$$e_c(X, Y, \theta_P) = \sin(\phi(\theta_P))\,(X - X_{\text{ref}}(\theta_P)) - \cos(\Omega(\theta_P))\,(Y - Y_{\text{ref}}(\theta_P)) \tag{2.13}$$

where $\phi(\theta_P)$ in Eq. (2.14) is the angle of the tangent to (X_{ref}, Y_{ref}) at the x-axis.

$$\phi(\theta_P) = \arctan\left(\frac{\partial Y_{\text{ref}}(\theta_P)}{\partial X_{\text{ref}}(\theta_P)}\right) \tag{2.14}$$

In practice, an approximation θ_A is made of θ_P. This introduces a lag error which is added in the cost function and also penalized. This is discussed further in [4]

2.3 Pacejka Dynamic Curvilinear Bicycle Model

The Pacejka Dynamic Curvilinear Bicycle Model presents a simple bicycle model in Fig. 2.3 with pacejka tire forces that captures the essential racing dynamics. The pacejka tire forces are a function of the front and rear slip angles α_F and α_F. See Eq. (2.16) The bicycle model in curvilinear coordinates presents that state dynamics in Eq. (2.15) in a reference frame local to the race car.

$$
\begin{aligned}
\dot{s} &= \tfrac{v_x \cos\mu - v_y \sin\mu}{1-n\kappa(s)}, \\
\dot{n} &= v_x \sin\mu + v_y \cos\mu, \\
\dot{\mu} &= r - \kappa(s)\dot{s}, \\
\dot{v}_x &= \tfrac{1}{m}\left(F_x - F_{y,F}\sin\delta + m v_y r\right), \\
\dot{v}_y &= \tfrac{1}{m}\left(F_{y,R} + F_{y,F}\cos\delta - m v_x r\right), \\
\dot{r} &= \tfrac{1}{I_z}\left(F_{y,F} l_F \cos\delta - F_{y,R} l_R + M_{tv}\right), \\
\dot{\delta} &= \Delta\delta, \\
\dot{T} &= \Delta T.
\end{aligned}
\tag{2.15}
$$

$$
\begin{aligned}
\alpha_F &= \arctan\left(\tfrac{v_y + l_F r}{v_x}\right) - \delta, \\
\alpha_R &= \arctan\left(\tfrac{v_y - l_R r}{v_x}\right), \\
F_{y,F} &= D_F \sin\left(C_F \arctan\left(B_F \alpha_F\right)\right), \\
F_{y,R} &= D_R \sin\left(C_R \arctan\left(B_R \alpha_R\right)\right),
\end{aligned}
\tag{2.16}
$$

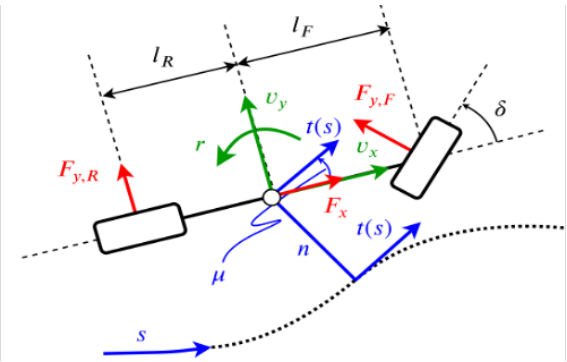


Figure 2.3: Cuvilinear Model

2.4 Path Contraints

Path constraints allow the the race car to stay within track limits. In curvilinear coordinates, since the state dynamics exist locally (in the car's reference frame), linear path constraints in the lateral deviation n can be used to keep the car within track limits. In Fig. 2.4, W_t is the track width, W_c is the car width, n is the lateral deviation from the reference point (X_{ref},Y_{ref}). With the centerline as the reference, the constraint follows as $-0.5(W_t + W_c) \leq n \leq 0.5(W_t + W_c)$

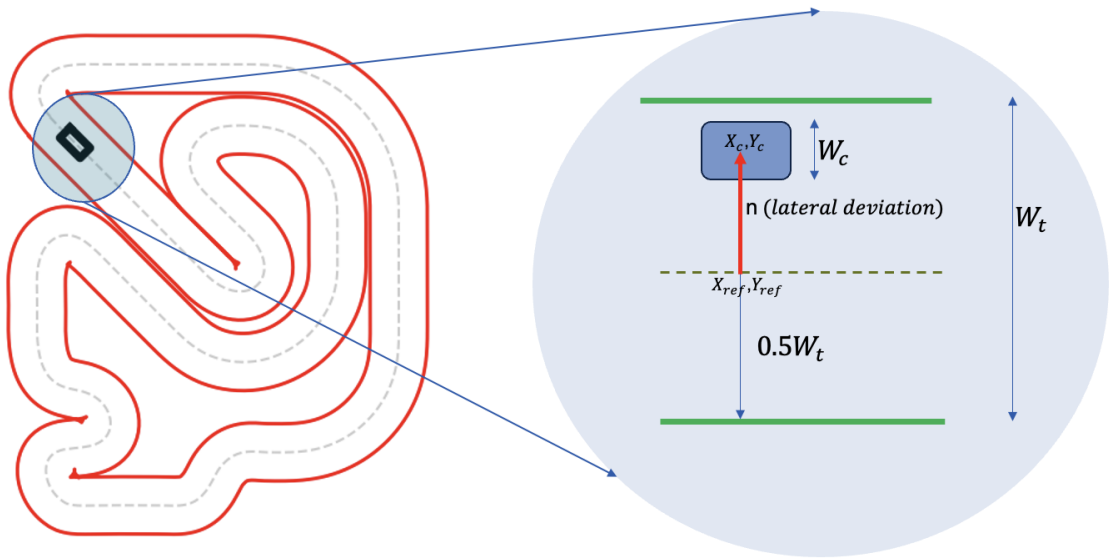


Figure 2.4: Curvilinear Path Constraint

Chapter 3

Approach

3.1 General Overview

The RL algorithm is trained offline and the value function learned by the critic is extracted. This is parameterized by an Artificial Neural Network. The influence of the value function as the terminal cost on the MPCC policy is investigated by selecting two different planning horizon lengths.

- A short horizon of 20 time-steps, where the influence of the terminal cost on the control policy is expected to be high.
- A long horizon of 30 time-steps, where the influence of the terminal cost is expected to be low. This is because the longer horizon is expected better capture the infinite horizon.

The problem is reformulated as

$$
\begin{aligned}
J^*_{\text{RL+MPCC}}(x(t)) = \min_{u_0,\ldots,u_{N-1}} \; & \sum_{k=t}^{t+N-1} h(x_k, u_k) + V^*_{\text{RL}}(x_{t+N}) \\
\text{subject to:} \quad & x_{k+1} = f(x_k, u_k), \quad \forall k \in \{t, \ldots, t+N-1\}, \\
& x_k \in \mathcal{X},\, u_k \in \mathcal{U}, \quad \forall k \in \{t, \ldots, t+N-1\}, \\
& x_N \in \mathcal{X}_N, \\
& x_0 = x(t).
\end{aligned}
\tag{3.1}
$$

where $h(x_k, u_k)$ is the stage cost, $V^*_{\text{RL}}(x_{t+N})$ is the terminal cost parameterized by a pytorch neural network learned by the critic and N is the horizon length. In practice, L4CasADi [8] was used to integrate the learned value function with ACADOS [11] which is the software primarily used for this work.

3.2 MPCC warm start using RL Policy

The idea of warm starting the solver used to solve the optimal control problem is beneficial in allowing the solver to easily find a feasible solution [11]. This can be

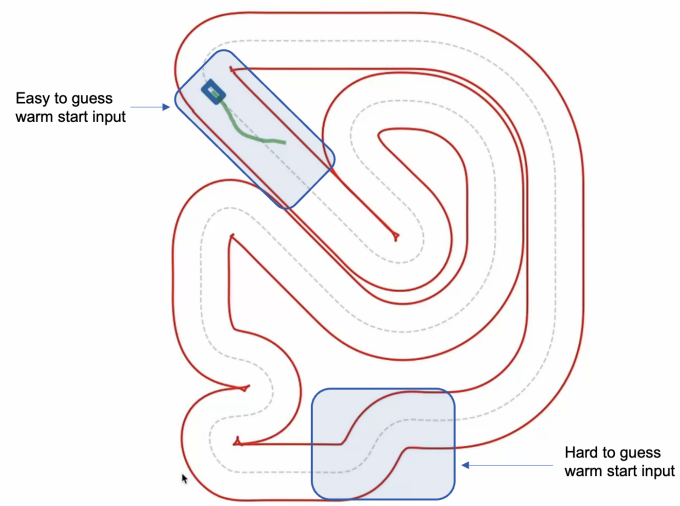


Figure 3.1: Race track showing warm-start regions

done by either using an initial guess of the decision or optimization variables for the length of the horizon or planning stages. This may be needed at the first step of the problem and in subsequent steps. For the latter, it can be done with a possibly shifted solution from the problem solution at the previous step. In this work, an approach was taken to warm-start the solver at the initial time step using the learned policy $\pi_\Omega(a|s)$. Note that the policy is a learned ANN. This was especially useful in hard-to-guess input regions like the chicane on the race track. See Fig. 3.1

3.2.1 Warm-start using an initial guess and dynamics

For the race track, there are some regions where warm starting the solver at the initial time step is easy. An example is the straight shown in Fig. 3.1. One can quickly guess a combination of throttle inputs T and steering angles δ for the entire planning horizon N which will ensure that the race car satisfies the path constraints for that entire horizon. For example, assuming the race started on a straight, a good input guess for the planning horizon can be half throttle and no zero steering input. This input sequence combination can be used to propagate the dynamics forward in time for the entire horizon length to get a good warm start solution for the solver. This can be updated for each planning stage (0 to N) in the solver. This is shown in Fig. 3.2

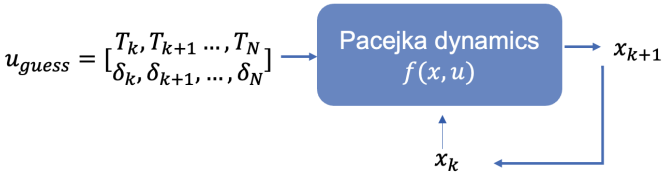


Figure 3.2: Warm-starting with initial guess

3.2.2 Warm-start using an Actor (RL policy) and dynamics

Assuming now that the race starts very close to a hard-to-guess input region like the chicane in Fig. 3.1 , a sequence combination ofthrottle and steering controls cannot be easily determined by guessing, such that that race car for the entire planning horizon does not violate the path constraint. To tackle this, the learned Actor or the RL policy is used to determine the control actions to propagate the dynamics by taking as argument the state as shown in Fig. 3.3

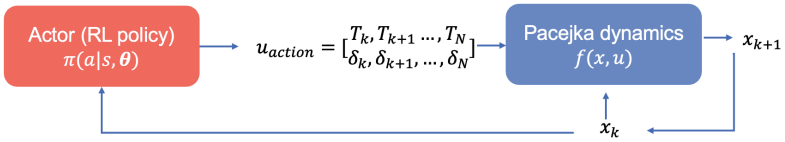


Figure 3.3: Warm-starting using the Actor (RL policy)

Chapter 4

Results

This chapter discusses the results after implementing the approach presented in chapter 3.

4.1 MPCC Policy with RL Terminal Cost

This part of the result discusses the behaviour of the MPCC policy under two cases.

- With the value function $V^*_{\mathrm{RL}}(x_{t+N})$ as terminal cost
- without the terminal cost

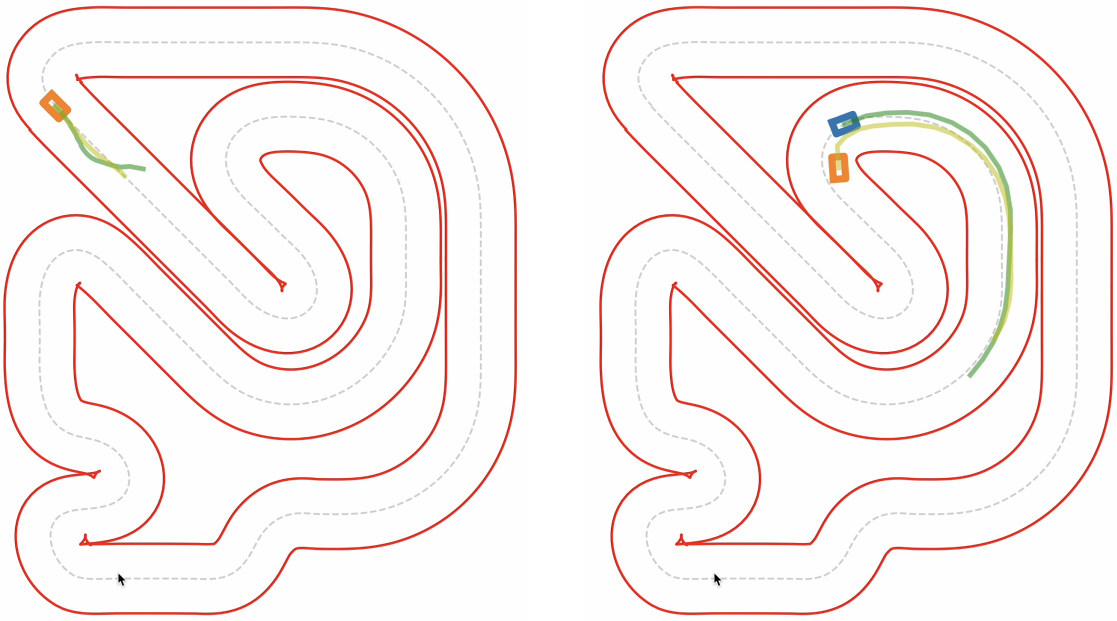
In both cases, the horizon N is adjusted to be 20 and 30 to investigate it influence as well.

4.1.1 Short horizon, N = 20

The planning horizon $N = 20$ is considered the short horizon. The expectation is that the influence of the terminal cost on the resulting MPCC control policy will be high and apparent. Therefore, the terminal cost is also expected to at least capture some part of the remaining horizon and control policy left over from solving a finite time problem. In Fig. 4.1, there are two instances of the racecar in the race. Both start with the same initial condition with the MPCC controller

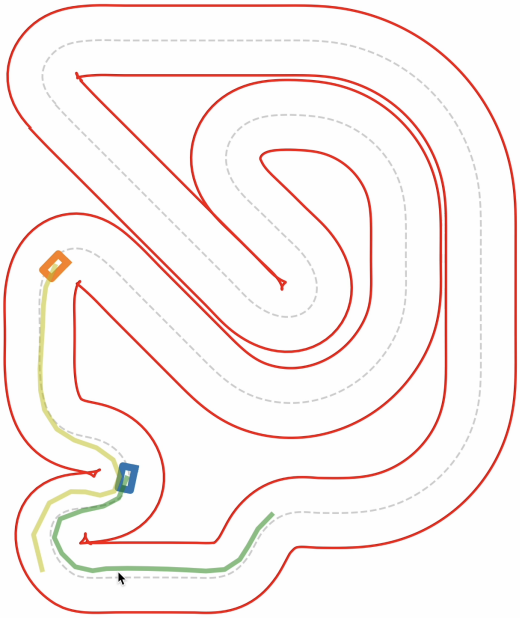
- The blue car has the terminal cost. (MPCC + $V^*_{\mathrm{RL}}(x_{t+N})$)
- The orange car has no terminal cost. (MPCC only)

It is at least obvious from Fig. 4.1 that as the race goes on, the car with the terminal cost influence gains an advantage and pulls away with high average lap velocities and faster lap times shown in Fig. 4.3 and Fig. 4.4
To summarize for horizon, $N = 20$, when;

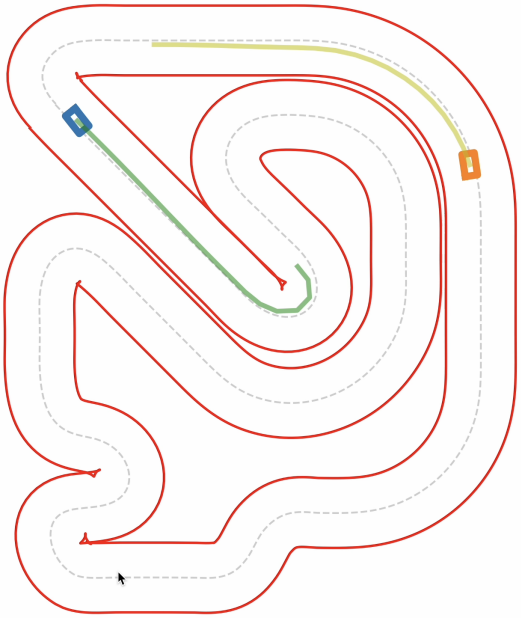


(a) t = 0s

(b) t = 12s



(c) t = 23s



(d) t = 35s

Figure 4.1: A comparison between the MPCC control policy with and without the terminal cost for N = 20.

RL value function is used as terminal cost

1. The car brakes a lot less in the corners and carries faster corner speeds as shown in Fig. 4.3

2. Consequently, it is able to get up to speed more quickly with less throttle as

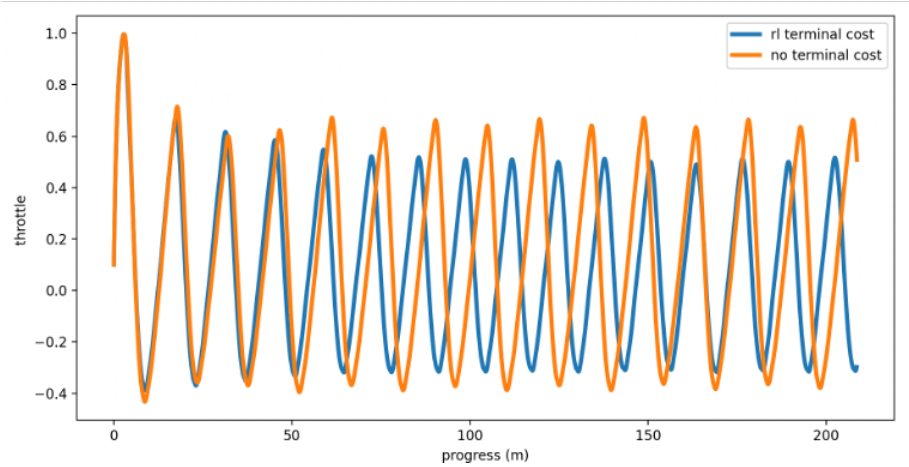


Figure 4.2: Race throttle profile

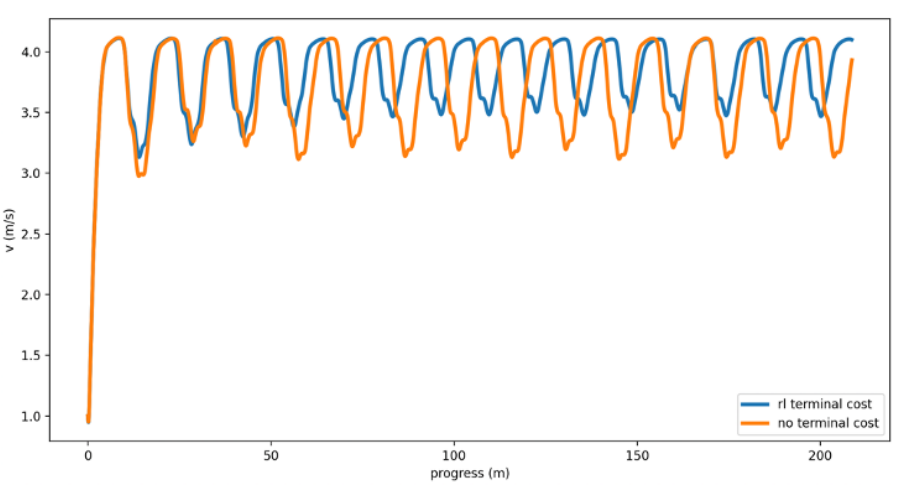


Figure 4.3: Race velocity profile

shown in Fig. 4.2

3. As a result it achieves faster lap times in Fig. 4.4.

There is no terminal cost

1. The car brakes a lot harder in the corners and carries slower corner speeds as shown in Fig. 4.3

2. Consequently, it tries to get up to speed with hard acceleration or throttle in Fig. 4.2

3. As a result it achieves slower lap times in Fig. 4.4

This shows a clear performance gain with the terminal cost by the controller.

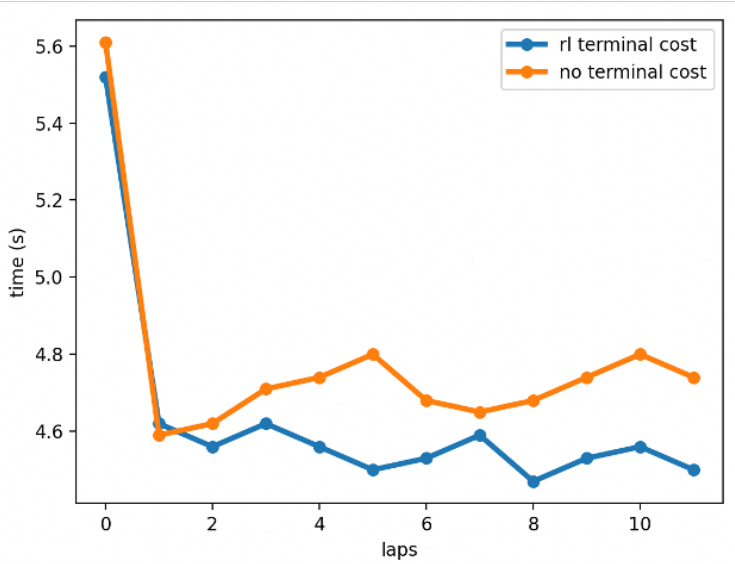


Figure 4.4: Lap times

4.1.2 Long horizon, N = 30

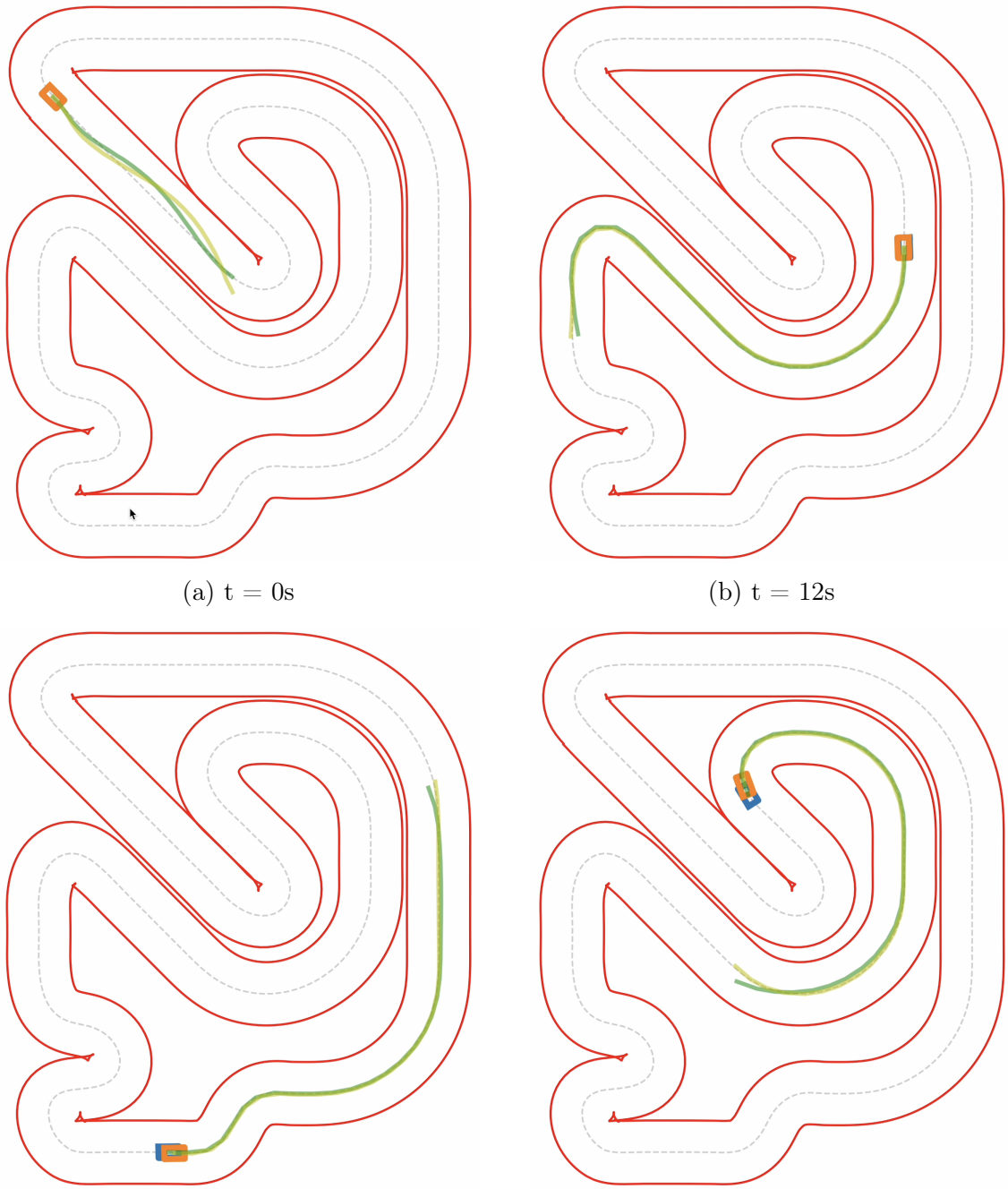
The planning horizon $N = 30$ is considered the long horizon. The influence of the terminal cost is expected to die down and be negligible in the control policy. This is because with a longer horizon, the controller is expected to better capture the infinite horizon therefore, the terminal cost should have little to no influence.
From the race in Fig. 4.5, the race velocity profile in Fig. 4.7 respectively and Fig. 4.8, a similar performance in the controller instances with and without the terminal cost is displayed. A demonstration of fairly similar and consistent race positions at all times in Fig. 4.5, lap times in Fig. 4.8, and velocities in Fig. 4.7.

4.1.3 Race Track Velocity Map

The race velocity track map shows the average velocities achieved by the two instances of the race car (with and without terminal cost) at different points on the track. This can be seen in Fig. 4.9. In the uncovered portions (outside the black triangle) reveal velocities higher in Fig. 4.9a with terminal cost and roughly lower velocities in Fig. 4.9b without terminal cost. The summary of performance metrics is in Table 4.1. Although the gain in lap time with the terminal cost is very marginal (0.15s), this is enough to pull away with time and many laps as seen in Fig. 4.1.

Table 4.1: Performance metrics with and without terminal cost.

Description	Unit	MPCC + $V^*_{\mathrm{RL}}(x_{t+N})$	MPCC only
Lap time	s	4.63	4.78
Average velocity	m/s	3.78	3.65



(a) t = 0s (b) t = 12s

(c) t = 23s (d) t = 35s

Figure 4.5: A comparison between the MPCC control policy with and without the terminal cost for N = 30

4.2 Computational Impact

The computation impact on the solver with and without the terminal cost for both horizon lengths (20 and 30) is shown in Fig. 4.10. The results in this figure is generally expected with higher computational cost incured for optimizing over the

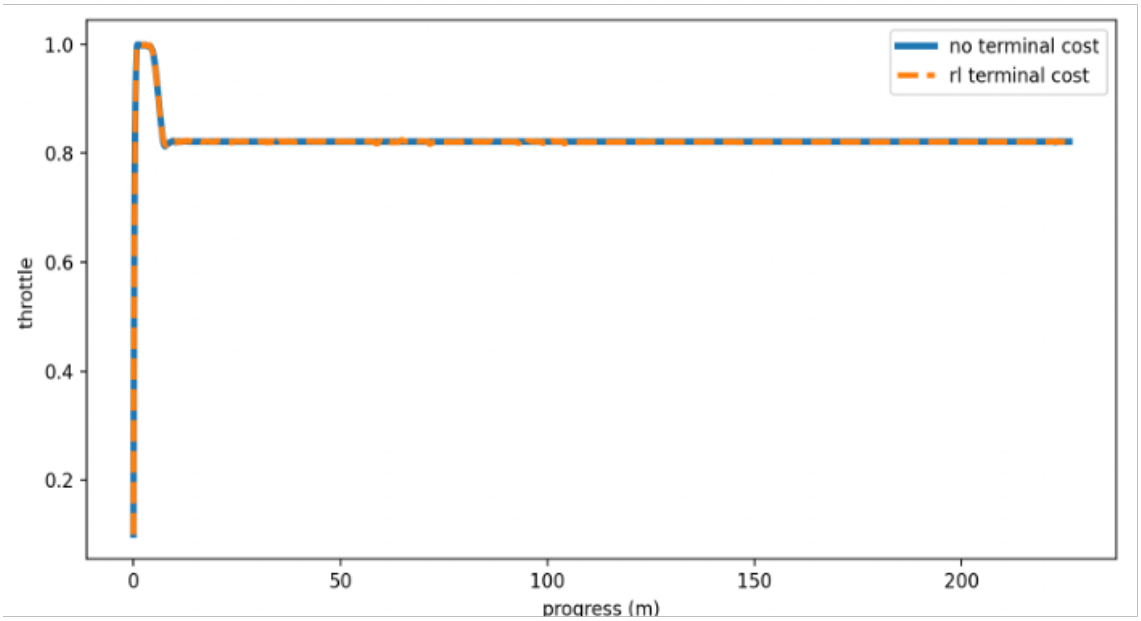


Figure 4.6: Race throttle profile

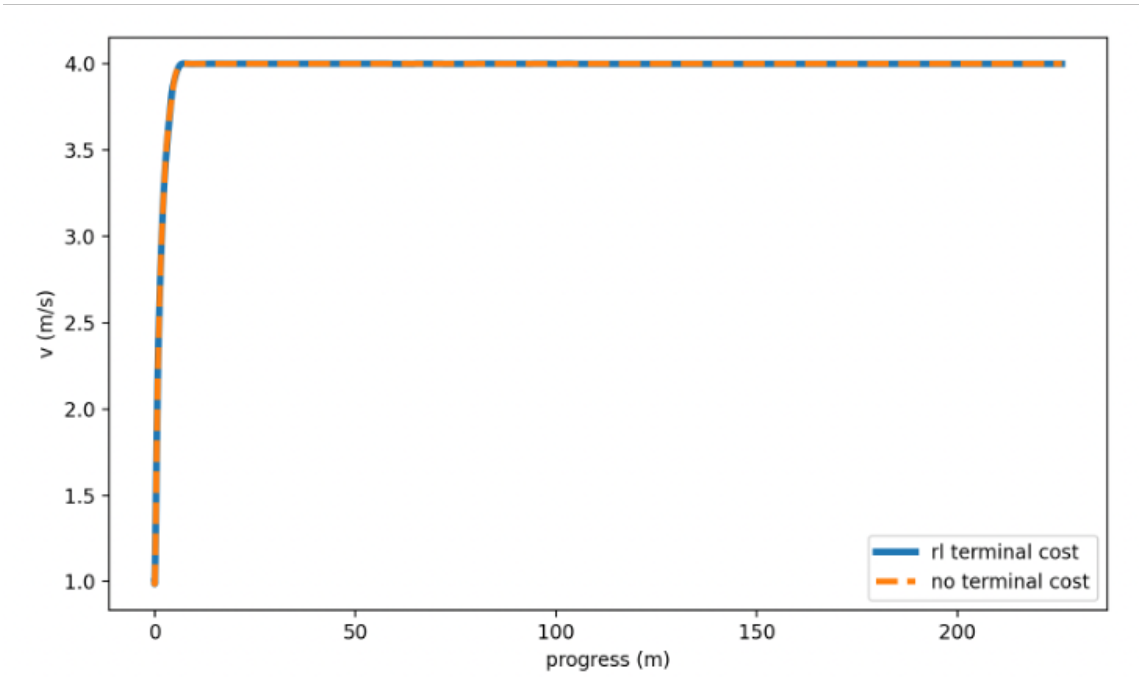


Figure 4.7: Race velocity profile

neural network terminal cost for both horizon lengths. Fig. 4.10 also reveals that, for no terminal cost, the compute cost is greater for N = 20 even though there are less decision variables to optimize compared with N = 30. Although this is counter intuitive, Fig. 4.11 reveals why.

1. For a horizon = 20, the controller is shortsighted and returns more suboptimal solution compared with horizon = 30.

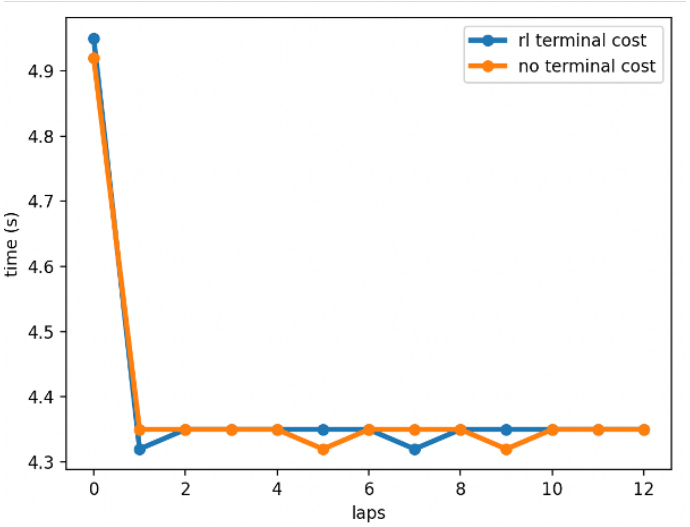
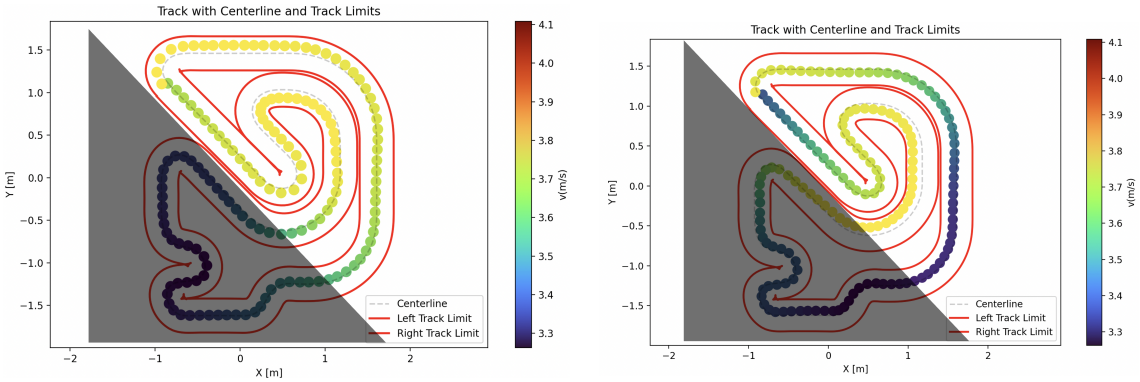


Figure 4.8: Lap times



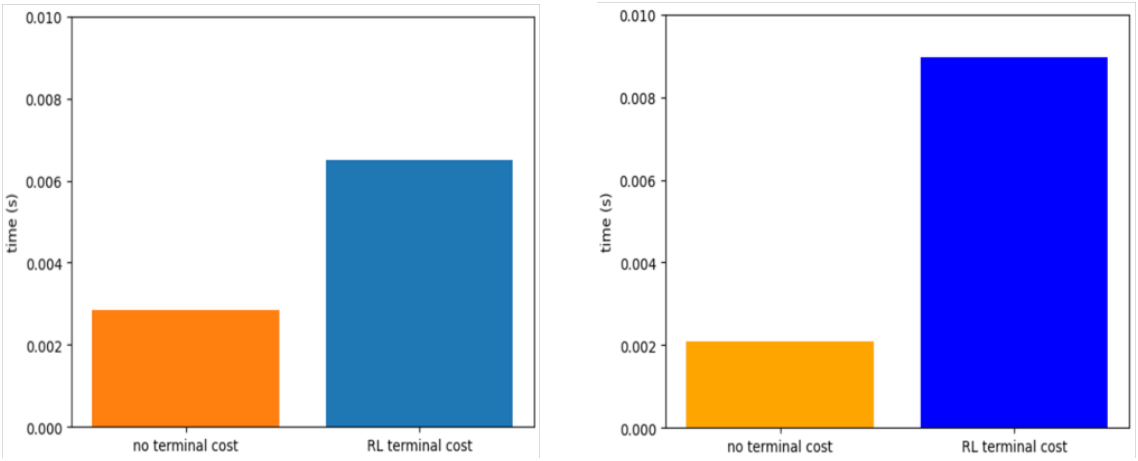
(a) terminal cost (b) no terminal cost

Figure 4.9: Average track velocity map for N = 20

2. This is because it cannot satisfy the set KKT tolerances in the solver within the maximum set iterations of 100.

3. Consequently it spends more time iterating through the SQP creating the extra computational overhead.

It is noteworthy that the compute time with terminal cost remains lower than the sampling time of 0.03s.



(a) compute time for N = 20 (b) compute time for N = 30

Figure 4.10: Computational performance

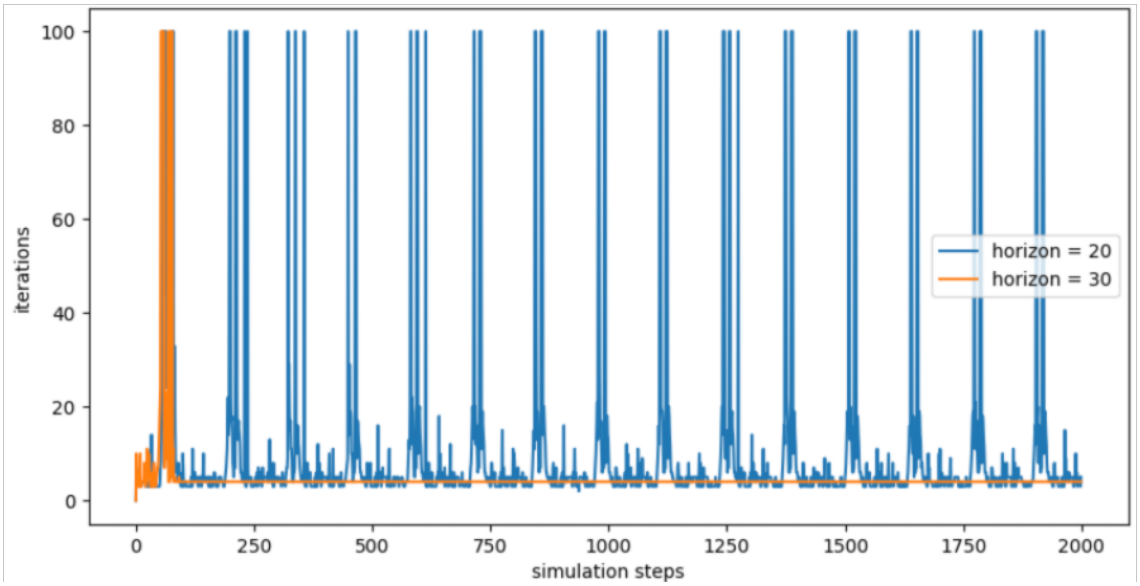


Figure 4.11: Number of iteration to return solver solution at each simulation step

Chapter 5

Conclusion

This work aimed to design terminal ingredients for a model predictive contouring controller with a specific focus on the terminal cost. The terminal cost was parameterized by a neural network and learned as the value function of an actor-critic reinforcement learning algorithm. Using L4CasADi [11] the value function was plugged into the formulated optimal control problem in ACADOS [8]. The study showed the influence of the terminal cost on the control policy which gave an advantage to the autonomous race car with this controller in terms of faster lap times and speed. Although the advantage was marginal in lap time, it was enough to pull away over the course of the race leading to better optimal driving behaviour. Noteworthy is that fact that the influence of the terminal cost on the control policy becomes apparent with shorter planning horizons. With longer planning horizons, it dies down and becomes neglible owing to a better approximation of the infinite horizon. Consequently, there was almost no difference between the controller with and without the terminal cost at $N = 30$. Owing to the the Pacejka model in curvilinear coordinates, the path constraints became linear in the lateral deviation. Finally, a case was made for warm starting the contouring controller in hard-to-guess input regions like the chicane on the race track.

Bibliography

[1] Nicolas Baumann, Edoardo Ghignone, Jonas KÃ¼hne, Niklas Bastuck, Jonathan Becker, Nadine Imholz, Tobias KrÃ¤nzlin, Tian Yi Lim, Michael LÃ¶tscher, Luca Schwarzenbach, Luca Tognoni, Christian Vogt, Andrea Carron, and Michele Magno. ForzaETH Race Stack–Scaled Autonomous Head-to-Head Racing on Fully Commercial Off-the-Shelf Hardware. *Journal of Field Robotics*, n/a(n/a). _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/rob.22429.

[2] Sabrina Bodmer, Lukas Vogel, Simon Muntwiler, Alexander Hansson, Tobias Bodewig, Jonas Wahlen, Melanie N. Zeilinger, and Andrea Carron. Optimization-Based System Identification and Moving Horizon Estimation Using Low-Cost Sensors for a Miniature Car-Like Robot, April 2024.

[3] Minghao Han, Lixian Zhang, Jun Wang, and Wei Pan. Actor-critic reinforcement learning for control with stability guarantee, 2020.

[4] Alexander Liniger, Alexander Domahidi, and Manfred Morari. Optimization-based autonomous racing of 1:43 scale RC cars. *Optimal Control Applications and Methods*, 36(5):628–647, 2015. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/oca.2123.

[5] Manish Prajapat, MojmÃr MutnÃ½, Melanie N. Zeilinger, and Andreas Krause. Submodular Reinforcement Learning, May 2024. arXiv:2307.13372 [cs].

[6] Rudolf Reiter, Andrea Ghezzi, Katrin BaumgÃ¤rtner, Jasper Hoffmann, Robert D. McAllister, and Moritz Diehl. AC4MPC: Actor-Critic Reinforcement Learning for Nonlinear Model Predictive Control, June 2024. arXiv:2406.03995 [eess].

[7] Ugo Rosolia and Francesco Borrelli. Learning How to Autonomously Race a Car: a Predictive Control Approach, October 2019. arXiv:1901.08184 [cs].

[8] Tim Salzmann, Jon Arrizabalaga, Joel Andersson, Marco Pavone, and Markus Ryll. Learning for CasADi: Data-driven Models in Numerical Optimization, December 2023. arXiv:2312.05873 [eess].

[9] Max Schwenzer, Muzaffer Ay, Thomas Bergs, and Dirk Abel. Review on model predictive control: an engineering perspective. *Int J Adv Manuf Technol*, 117(5):1327–1349, November 2021.

[10] Bastian SchÃ¼rmann, Moritz Klischat, Niklas Kochdumper, and Matthias Althoff. Formal safety net control using backward reachability analysis. *IEEE Transactions on Automatic Control*, 67(11):5698–5713, 2022.

[11] Robin Verschueren, Gianluca Frison, Dimitris Kouzoupis, Jonathan Frey, Niels van Duijkeren, Andrea Zanelli, Branimir Novoselnik, Thivaharan Albin, Rien Quirynen, and Moritz Diehl. acadosâa modular open-source framework for fast embedded optimal control. *Math. Prog. Comp.*, 14(1):147–183, March 2022.

[12] Ruofan Wu, Junmin Zhong, and Jennie Si. Actor-critic reinforcement learning with phased actor, 2024.

ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Institute for Dynamic Systems and Control
Prof. Dr. R. D'Andrea, Prof. Dr. L. Guzzella

Title of work:

On the Design of Terminal Ingredients for MPCC (using RL)

Thesis type and date:

Master Thesis, December 2024

Supervision:

Dr. Andrea Carron
Prof. Dr. Melanie Zeilinger

Student:

Name:	William Akuffo
E-mail:	wakuffo@student.ethz.ch
Legi-Nr.:	97-906-739
Semester:	5

Statement regarding plagiarism:

By signing this statement, I affirm that I have read and signed the Declaration of Originality, independently produced this paper, and adhered to the general practice of source citation in this subject-area.

Declaration of Originality:

`http://www.ethz.ch/faculty/exams/plagiarism/confirmation_en.pdf`

Zurich, 20. 12. 2024: _____